МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

УНИВЕРСИТЕТ ИТМО

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ 4

Apache ZooKeeper

Обеспечение отказоустойчивости и защита от устаревших записей

Вариант 1: программист, разработчик на 1С

Выполнил:
Карташов Игорь Олегович
Группа J3312

Санкт-Петербург
2026

1. Развёртывание ансамбля

Развернул три копии Apache ZooKeeper 3.9.6 на Windows с Java 17 (Temurin 17.0.20.1+1). В JAVA_HOME указал папку runtime/jdk-17.0.20.1+1. Каждую копию запускал штатной командой bin\zkServer.cmd из её каталога servers/zkN.

Сервер / myid	Клиент	Кворум	Выборы	dataDir
zk1 / 1	2181	2888	3888	../zk1/data
zk2 / 2	2182	2889	3889	../zk2/data
zk3 / 3	2183	2890	3890	../zk3/data

В conf/zoo.cfg у всех трёх серверов одинаковый список участников. Каталоги данных отдельные, в каждом лежит свой файл myid с числом 1, 2 или 3. Для сессий задан тайм-аут 10 секунд; tickTime равен 2000 мс.

```
server.1=127.0.0.1:2888:3888
server.2=127.0.0.1:2889:3889
server.3=127.0.0.1:2890:3890
```

Проверил роли командой srvr: zk3 стал лидером, zk1 и zk2 - последователями. Все три процесса работают на одном ПК; в опытах проверял отказ отдельных процессов.

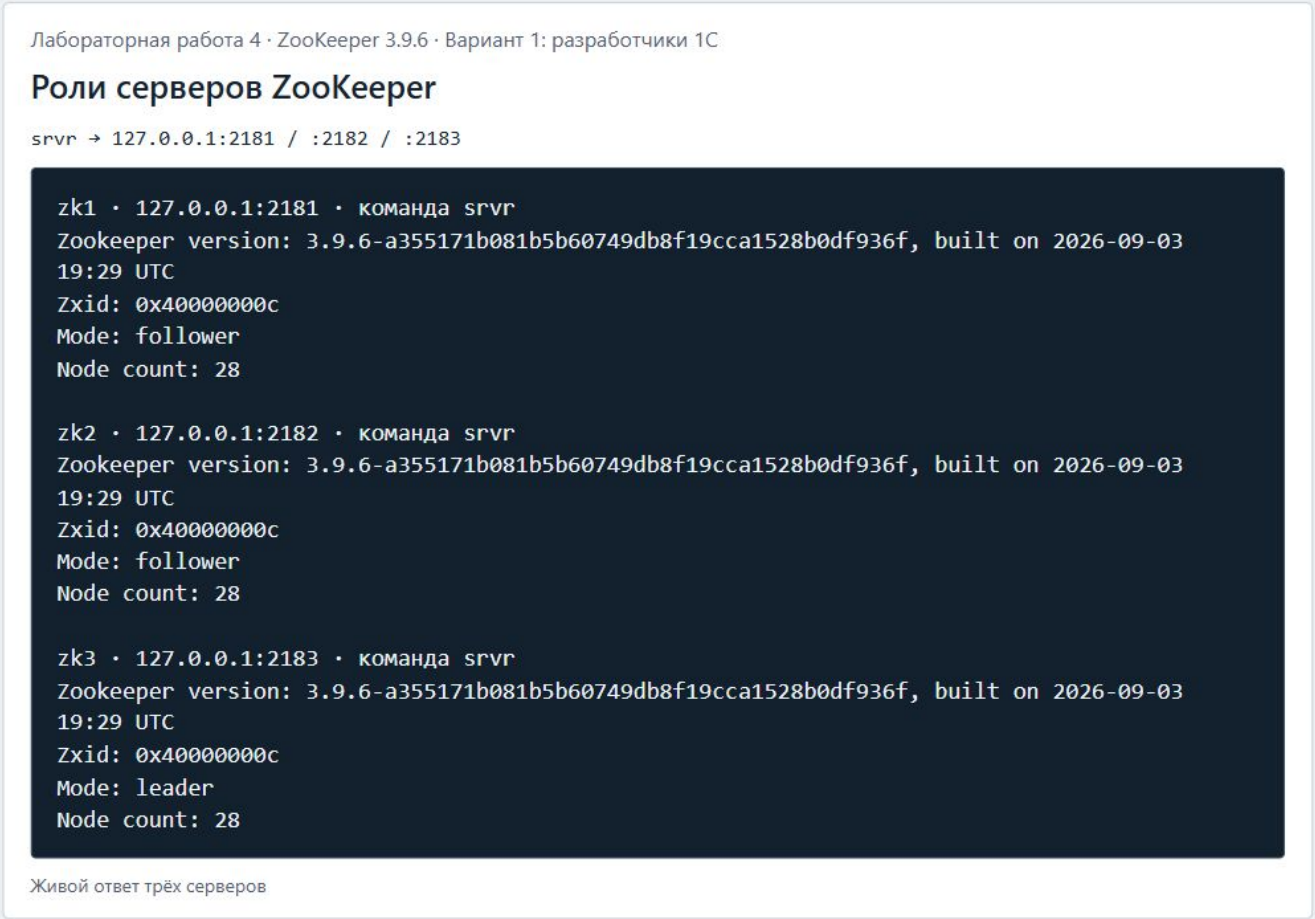


Рисунок 1. Ответы srvr трёх серверов в локальной консоли

2. Потеря одного и двух серверов

Сначала остановил zk1. Оставшиеся zk2 и zk3 сохранили кворум: новое значение записалось и прочиталось обратно. Затем остановил zk2. Процесс zk3 продолжал работать, но перестал обслуживать запросы ZooKeeper.

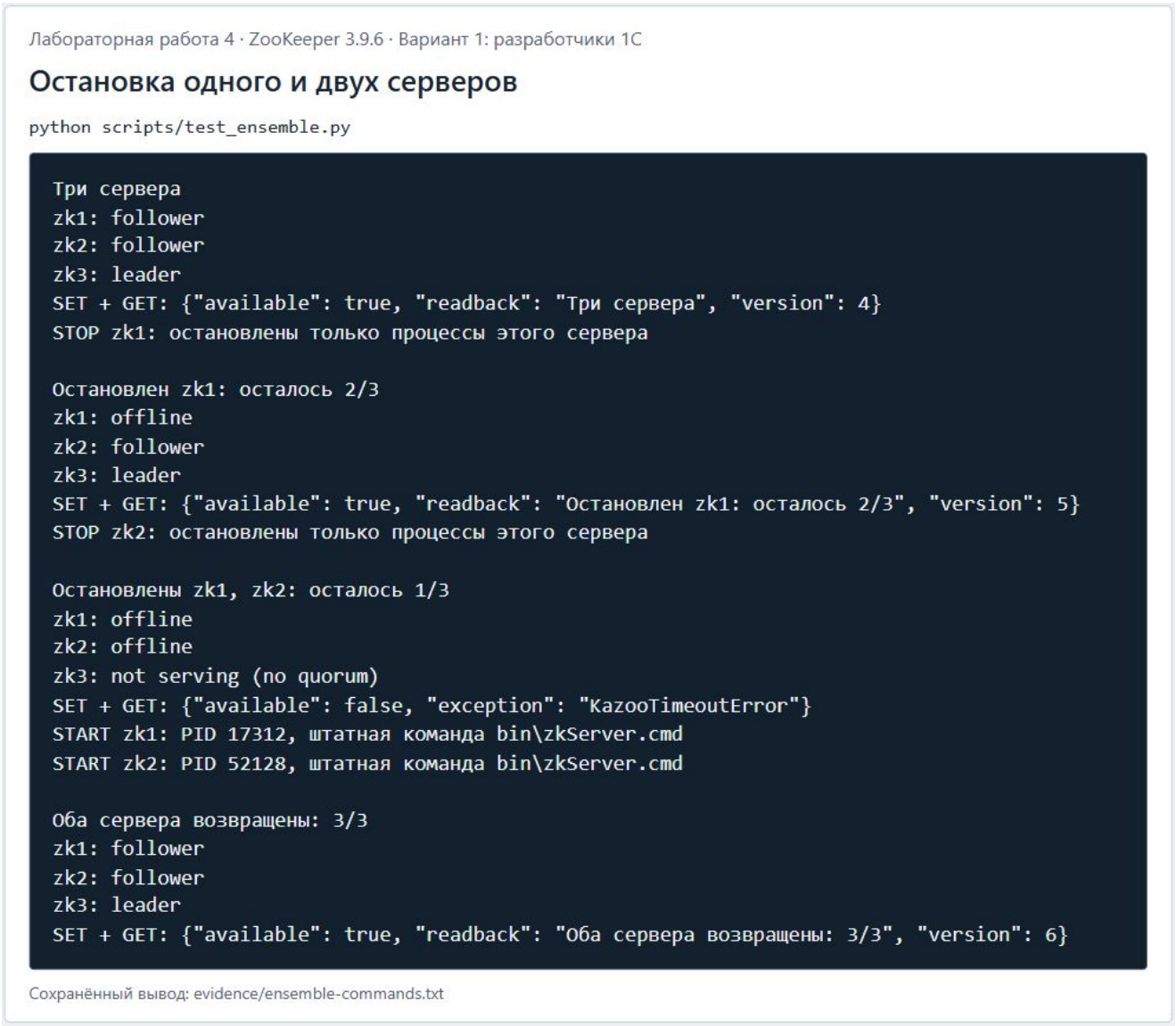


Рисунок 2. Проверка записи и чтения при 3, 2 и 1 работающем сервере

При одном оставшемся сервере srvr вернул «This ZooKeeper instance is not currently serving requests», а попытка подключения клиента завершилась KazooTimeoutError. После запуска zk1 и zk2 снова появились один лидер и два последователя, запись и чтение восстановились.

Для трёх участников кворум равен двум. Поэтому потеря одного сервера допустима, а потеря двух останавливает обслуживание запросов.

3. Эфемерный узел

В zkCli создал узел командой create -e и проверил его через get -s из другого клиента. Поле ephemeralOwner было ненулевым: 0x20005e957750002. После команды quit в клиенте-владельце повторный запуск get -s вернул Node does not exist.

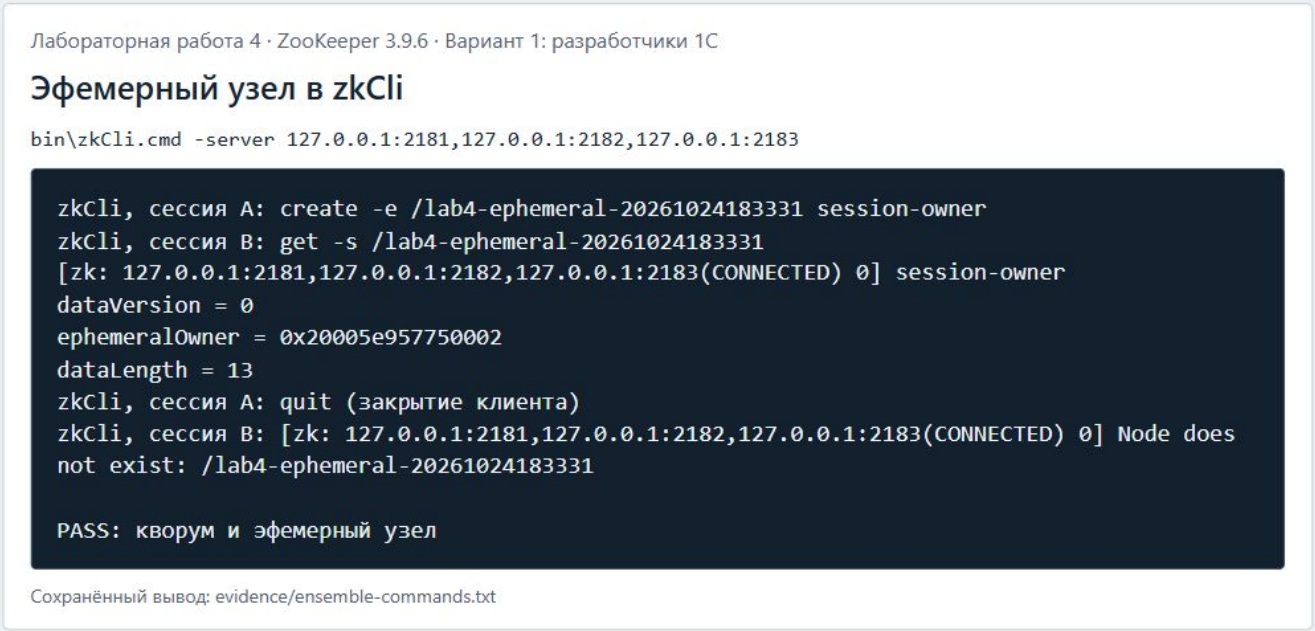


Рисунок 3. Узел существует до закрытия сессии и исчезает после него

4. Координатор расчёта пар навыков

Использовал тот же CSV из s3://hh-lake/raw/it_vacancies_2024.csv и вариант 1C из работы 3. Отбирал роль «Программист, разработчик» и названия с 1C/1C и словами «программист», «разработчик», developer или programmer. Битрикс/Bitrix и Java Developer исключил. Навыки приводил к нижнему регистру, убирал пробелы по краям и повторы внутри вакансии, затем считал неупорядоченные пары.

Запустил два отдельных процесса A и B. Лидера выбирает Kazoo Lock на эфемерных последовательных узлах. Для обоих клиентов сервер подтвердил тайм-аут сессии 10 секунд. При получении лидерства координатор увеличивает эпоху.

Через каждые 10 000 строк координатор сохраняет в S3 счётчики пар, список учтённых вакансий и позицию в CSV. В ZooKeeper хранится ссылка на чекпоинт и его SHA-256. Запасной процесс загружает счётчики, пропускает уже учтённые строки и считает оставшуюся часть. В обоих опытах он восстановился с 20 000 строк и 5800 уникальных пар.

Лидера приостанавливал средствами ОС. Возобновлял A после завершения B, выдержав паузу не меньше 40 секунд. Между пакетами по 1000 строк добавлена задержка 30 мс, чтобы расчёт не успел закончиться до паузы.

4.1. Проверка окружения и журнал координатора

Проверил JAVA_HOME, параметры zoo.cfg и файлы myid. В журнале B видно, как он получил лидерство, загрузил чекпоинт и продолжил обработку следующих строк.



Рисунок 4. Проверка Java 17 и настроек трёх копий ZooKeeper

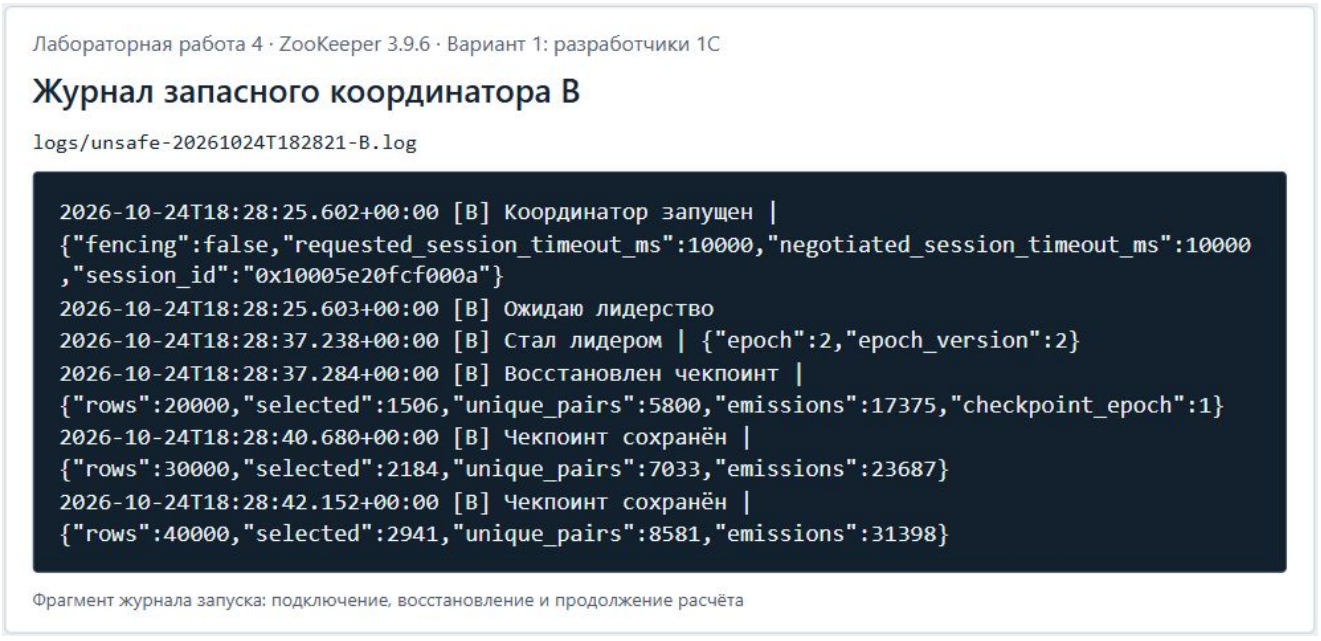


Рисунок 5. Журнал B: восстановление 20 000 строк и следующие чекпоинты

5. Приостановка без защиты: FENCING=0

После чекпоинта на 20 000 строк приостановил A. Через 11,576 с B стал лидером эпохи 2, восстановил состояние и записал результат. Через 40,194 с возобновил A: он завершил свой расчёт и тоже записал результат.

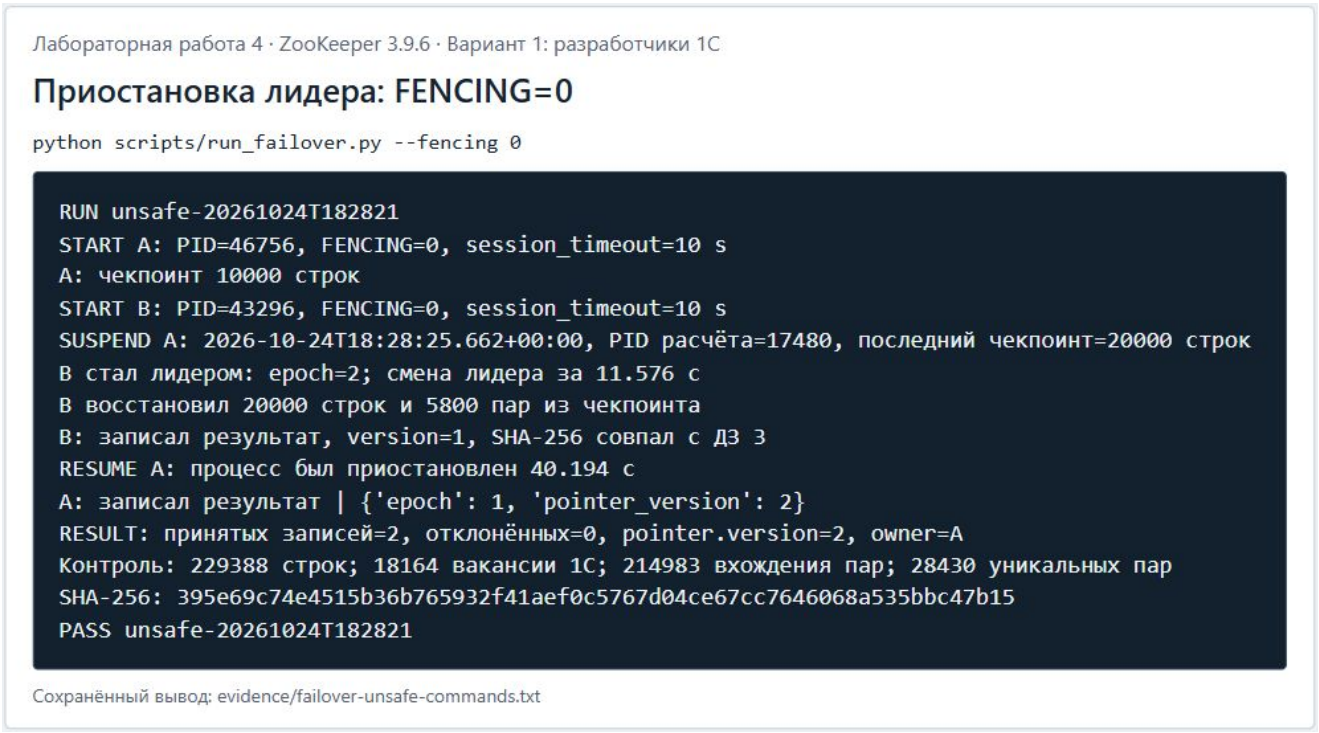


Рисунок 6. Две успешные публикации: сначала B, затем возобновлённый A



Рисунок 7. get -s: указатель версии 2 перезаписан координатором A, эпоха 1

B увеличил версию указателя с 0 до 1. Возобновлённый A повысил её до 2 и вернул указатель на старую эпоху.

6. Приостановка с защитой: FENCING=1

Повторил опыт с проверкой эпохи при публикации указателя. Проверка версии узла epoch и изменение узла result выполняются одной транзакцией ZooKeeper. После получения лидерства B эпоха стала равна 2, поэтому токен A из эпохи 1 устарел.

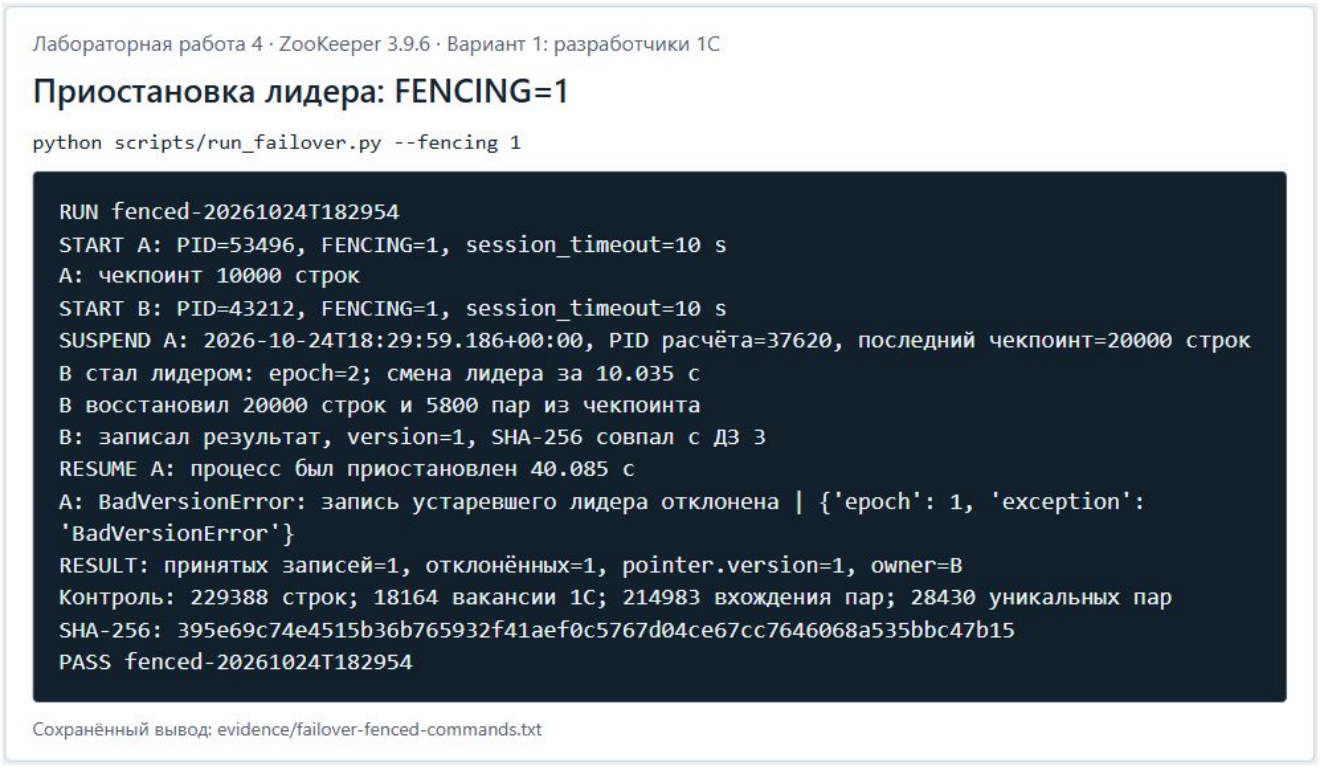


Рисунок 8. После возобновления A сервер отклонил его запись с BadVersionError



Рисунок 9. get -s: версия указателя осталась 1, автор B, эпоха 2

A не смог заменить указатель B. Файлы эпох в S3 имеют разные имена; опубликованным считается файл из узла result.

7. Сверка результатов и время смены лидера

Сравнил результаты A и B в обоих опытах с файлом skills_2.csv из работы 3. Совпали все ключи, частоты и байты файлов. Дополнительно скачал из S3 файл по каждому итоговому указателю и получил то же содержимое.

Показатель	Работа 3	FENCING=0	FENCING=1
Строк CSV	229 388	229 388	229 388
Вакансий 1С	18 164	18 164	18 164
Вхождений пар	214 983	214 983	214 983
Уникальных пар	28 430	28 430	28 430

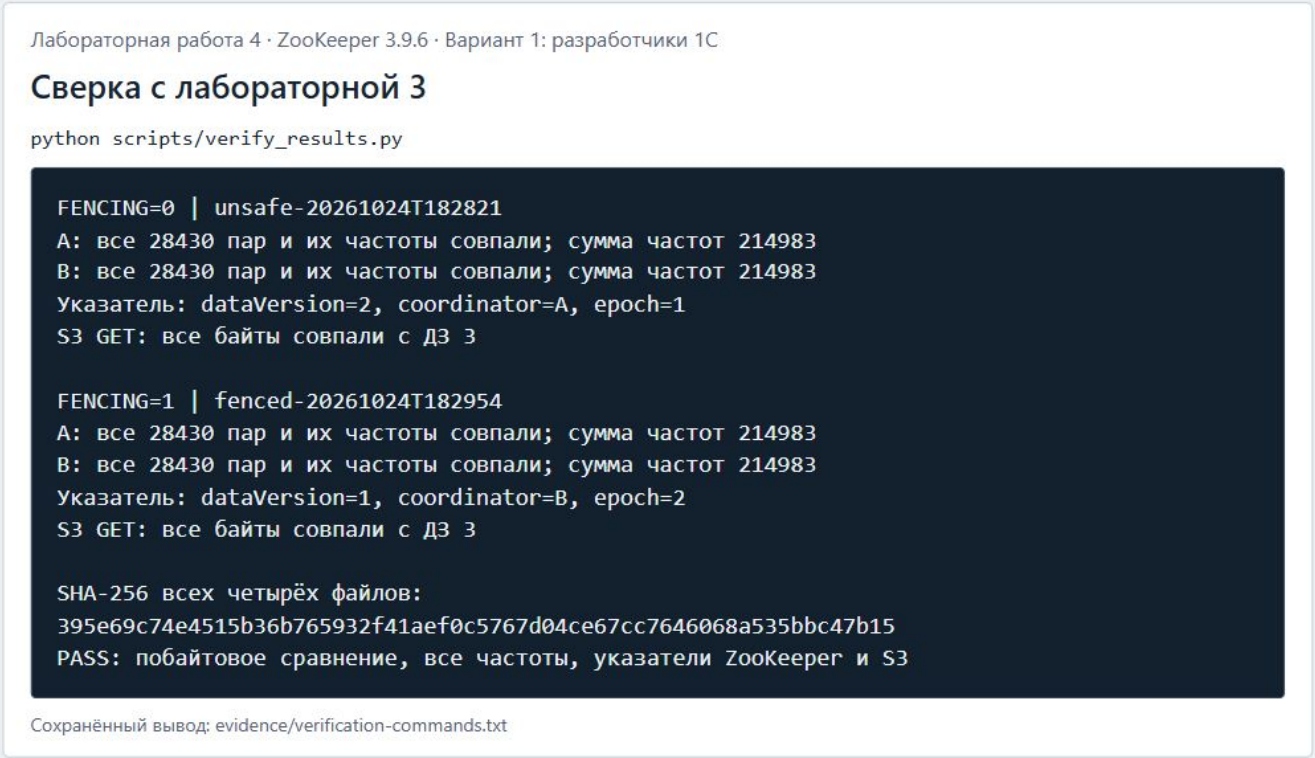


Рисунок 10. Проверка четырёх результатов, всех частот и чтения из S3

Время считал по монотонным часам: t смены = t получения лидерства B - t приостановки A. Начальная отметка сделана сразу после остановки вычисляющего процесса средствами ОС.

Измерение	Без защиты	С защитой
Смена лидера	11,576 с	10,035 с
Пауза A	40,194 с	40,085 с
Принятых публикаций	2	1
Итоговый указатель	версия 2, автор A	версия 1, автор B

8. Ответы на вопросы

Почему три сервера, а не два или четыре?

ZooKeeper нужна работающая большая часть ансамбля. Кворум равен floor(N/2) + 1. При двух серверах нужны оба: потеря любого из них сразу останавливает запись. У трёх серверов достаточно двух, поэтому один можно потерять. Четвёртый сервер не даёт ещё одного допустимого отказа: для кворума всё равно нужны три.

Серверов N	Кворум	Допустимо отказов
2	2	0
3	2	1
4	3	1
5	3	2

Если требуется пережить два отказа, нужен ансамбль из пяти серверов. Размещать их тогда надо на разных машинах, иначе отказ одного ПК остановит весь ансамбль.

Почему эпоху проверяет получатель записи, а не ZooKeeper?

ZooKeeper может завершить старую сессию и выбрать нового лидера, но он не останавливает вычисляющий процесс. После паузы старый лидер продолжает выполнение с того места, на котором остановился, и может отправить запись со старой эпохой. Именно это произошло с A.

Поэтому получатель должен проверять эпоху в момент принятия записи. В моей реализации получатель публикует указатель в ZooKeeper и явно использует его проверку версии: check(epoch) и set_data(result) входят в одну транзакцию. Сам ZooKeeper умеет выполнить такую проверку, но приложение должно её запросить. Произвольную запись напрямую в S3 или другую базу он автоматически не защищает.

Вывод

Ансамбль продолжил работу при потере одного сервера и потерял кворум при потере двух. Чекпоинт позволил запасному координатору завершить расчёт с теми же данными. Без fencing старый лидер повторно опубликовал результат, а с проверкой эпохи его запись была отклонена и указатель запасного координатора сохранился.

Конфигурации, код, журналы запусков и отчёт: github.com/IGK-arch/itmo-zookeeper-1c.